R.M.D. ENGINEERING COLLEGE

22IT201 – DATABASE MANAGEMENT SYSTEMS

UNIT – 1

DATABASE CONCEPTS

COURSE OBJECTIVES

- To understand the basic concepts of Data modeling and Database Systems.
- To understand SQL and effective relational database design concepts.
- To learn relational algebra, calculus and normalization
- To know the fundamental concepts of transaction processing, concurrency control techniques, recovery procedure and data storage techniques.
- To understand query processing, efficient data querying and advanced databases.

SYLLABUS – UNIT 1

UNIT - I DATABASE CONCEPTS

Concept of Database and Overview of DBMS - Characteristics of databases –Data Models, Schemas and Instances - Three-Schema Architecture - Database Languages and Interfaces- Introductions to data models types- ER Model- ER Diagrams – Enhanced ER Model - reducing ER to table Applications: ER model of University Database Application – Relational Database Design by ER- and EER-to-Relational Mapping.

List of Exercise/Experiments

Case Study using real life database applications anyone from the following list

a) Inventory Management for a EMart Grocery Shop

b) Society Financial Management

c) Cop Friendly App – Eseva

d) Property Management – eMall

e) Star Small and Medium Banking and Finance

Build Entity Model diagram. The diagram should align with the business and functional goals stated in the application.

COURSE OUTCOMES

CO1: Map ER model to Relational model to perform database design effectively.

CO2: Implement SQL and effective relational database design concepts.

CO3: Apply relational algebra, calculus and normalization techniques in database design.

CO4: Understand the concepts of transaction processing, concurrency control, recovery procedure and data storage techniques.

CO5: Apply query optimization techniques and understand advanced databases.

CONCEPT OF DATABASE AND OVERVIEW OF DBMS

Definition of Database:

- A Database is a *collection of related data*.
- Database systems are designed to *manage large bodies of information*
- The Database System must
    - Ensure safety of information stored
    - Prevents unauthorized access

CONCEPT OF DATABASE AND OVERVIEW OF DBMS

- **Definition of Database Management System (DBMS):**

- A Database Management System *is a collection of interrelated data and a set of programs to access these data*.
- The DBMS is a *general-purpose software system* that facilitates the processes of defining, constructing, manipulating, and sharing databases among various users and applications.
- The primary goal of DBMS is to provide a way to *store and retrieve* database information in a convenient and efficient manner.
- It provides a *secure and survivable medium* for storing the data.
- Examples of DBMS : MySQL, Oracle, Microsoft SQL Server, Microsoft Access etc.,

CONCEPT OF DATABASE AND OVERVIEW OF DBMS

- **Database System Environment:**

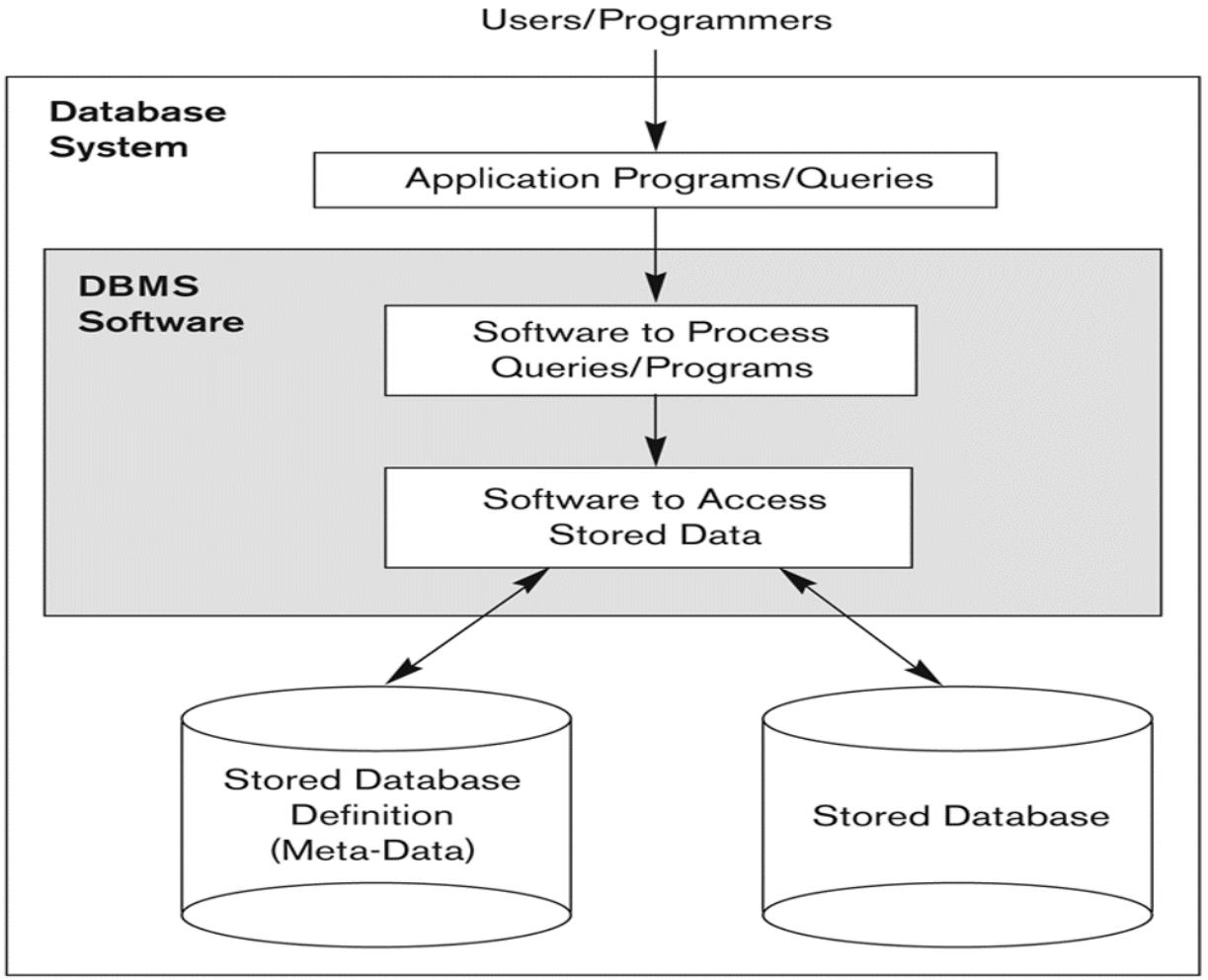


Figure: A Simplified Database System Environment

CONCEPT OF DATABASE AND OVERVIEW OF DBMS

- **Applications of Database Systems**

➢Banking
➢Universities
➢Online Retailers
➢Credit card transactions
➢Finance
➢Telecommunications
➢Research
➢Human Resources
➢Sales and Accounting
➢Airlines
➢Railways
➢Manufacturing industries

CONCEPT OF DATABASE AND OVERVIEW OF DBMS

Comparison of File Processing System and Database Systems:

- **Disadvantages of File Systems:**

1. Redundancy and Inconsistency
2. Difficulty in accessing data
3. Data Isolation
4. Integrity problems
5. Atomicity problems
6. Concurrent Access Anomalies
7. Security problems

CONCEPT OF DATABASE AND OVERVIEW OF DBMS

- **Advantages of File Systems**

File systems approach is suited for:

1. Simple, well-defined database applications that are not expected to change at all.
2. Embedded systems with limited storage capacity
3. No multiuser access to data is required.

CONCEPT OF DATABASE AND OVERVIEW OF DBMS

- **Advantages of using Database Approach**

1. Controlling Redundancy
2. Restricting Unauthorized Access
3. Providing Persistent Storage for Program Objects and Data Structures
4. Providing Multiple User Interfaces
5. Representing Complex Relationships among Data
6. Enforcing Integrity Constraints
7. Providing Backup and Recovery

CONCEPT OF DATABASE AND OVERVIEW OF DBMS

- **Disadvantages of Database Systems**

1. High initial investment in hardware, software and training
2. The generality that a DBMS provides for defining & processing data.
3. Overhead for providing security, concurrency control, recovery and integrity functions.

CHARACTERISTICS OF DATABASE

1. Self Describing Nature of a Database System:

- The database system not only contains the database, but also a complete definition and description of the database structure and constraints. This definition is stored in the Database catalog
- The information stored in the catalog is called metadata and this separates data and description part of the database
- In traditional File processing, data definition is a part of application programs themselves

CHARACTERISTICS OF DATABASE

2. INSULATION BETWEEN PROGRAM AND DATA & DATA ABSTRACTION:

❑Program-Data Independence

- In DBMS, the structure of data files is stored in the DBMS catalog separately from the access programs.
- In File, the structure of data files is embedded in the application programs that access that file. So, any changes to the structure of the file may require changing all programs that access that file.

❑Program-Operation Independence

- User application programs can operate on the data by invoking these operations through their names and arguments, regardless of how the operations are implemented.
- This may be termed program-operation independence.

The characteristic that allows program-data independence and program-operation independence is called data abstraction.

CHARACTERISTICS OF DATABASE

3. SUPPORT OF MULTIPLE VIEWS OF THE DATA:

- A database typically has many users, each of whom may require a different perspective or view of the database.
- A view may be a subset of the database or it may contain virtual data that is derived from the database files but is not explicitly stored.
- A multiuser DBMS whose users have a variety of applications must provide facilities for defining multiple views.

CHARACTERISTICS OF DATABASE

4. SHARING OF DATA AND MULTIUSER TRANSACTION PROCESSING:

- A multiuser DBMS must allow multiple users to access the database at the same time.
- The DBMS must include concurrency control software to ensure that several users trying to update the same data do so in a controlled manner so that the result of the updates is correct.

SCHEMAS AND INSTANCES

Database Schema: The overall design of the database is called as database schema. It is specified during database design and is not expected to change frequently.

- **Physical Schema:** Describes the database design at physical level
- **Logical Schema:** Describes the database design at logical level
- **Sub Schema:** Describes different views of the database. A database may have several schemas at the view level

Database Instance: The collection of information stored in the database at a particular moment of time is known as database instance or database snapshot.

SCHEMAS AND INSTANCES

DATA INDEPENDENCE:

- Data independence, which can be defined as the capacity to change the schema at one level of a database system without having to change the schema at the next higher level.

TYPES OF DATA INDEPENDENCE:

1. Logical Data Independence

- Logical data independence is the capacity to change the conceptual schema without having to change external schemas or application programs.

2. Physical Data Independence

- Physical data independence is the capacity to change the internal schema without having to change the conceptual schema.

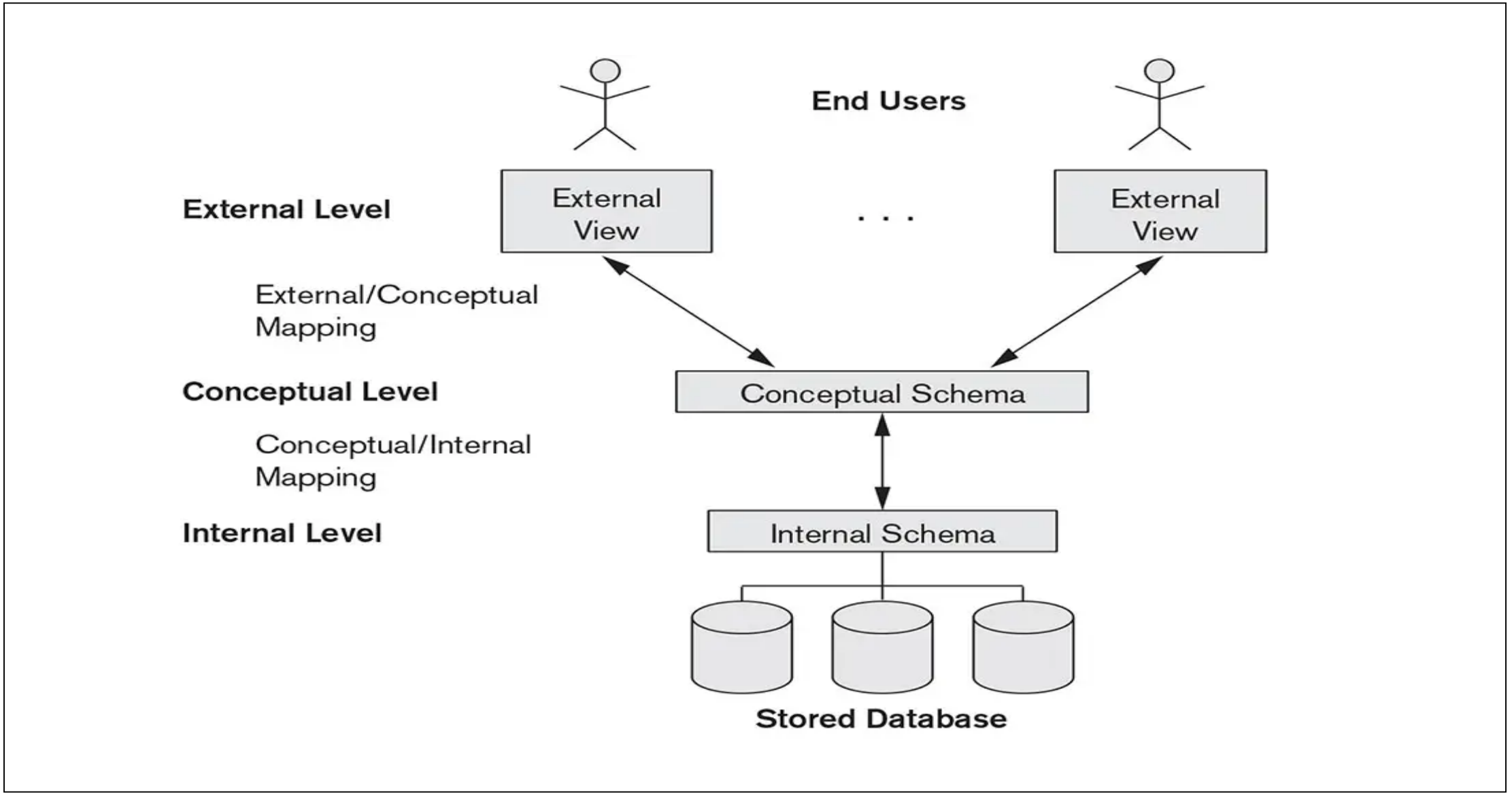
THREE SCHEMA ARCHITECTURE

- A database system is a collection of interrelated data and a set of programs that allow the users to access and modify these data.
- A major purpose of the database system is to provide users with an abstract view of the data (i.e.) the system hides details of how the data are stored and maintained.
- The goal of the three-schema architecture is to separate the user applications from the physical database

THREE SCHEMA ARCHITECTURE

- In this architecture, schemas can be defined at the following three levels:
- Internal level / Physical level
- Conceptual level / Logical level
- External level / View level

THREE SCHEMA ARCHITECTURE



THREE SCHEMA ARCHITECTURE

1. Internal level / Physical level:

- Describes how the data are stored.
- It is the lowest level of abstraction.
- The physical level describes complex low level data structures and access paths for the database in detail.

THREE SCHEMA ARCHITECTURE

2. Conceptual level / Logical level:

- Describes what data are stored in the database and what relationship exists among these data.
- It is the next higher level of abstraction.
- The conceptual schema hides the details of physical storage structures and concentrates on describing entities, data types, relationships, user operations, and constraints.
- The user of the logical level does not need to be aware of complex physical-level structures. This is referred as physical data independence.

THREE SCHEMA ARCHITECTURE

3. External level / View level:

- Describes only part of the entire database.
- It includes a number of external schemas or user views.

Example:

Consider the ‘instructor’ relation

Instructor (id, name, dept, salary)

THREE SCHEMA ARCHITECTURE

In Physical level,

- The records of the relation can be described as a *block of consecutive storage locations.*
- The Database System hides many of the lowest level storage details from the database programmers. The database administrators may be aware of certain details of physical organization of the data.

In Logical level,

- Each instructor record is described by a type definition

Example:

type instructor = record

ID : char (5);

Name : char(20);

Dept : char(20);

Salary : numeric (8,2);

end;

- Programmers and database administrators usually work at this level of abstraction

In View level,

- Several views of the database are defined.

DBMS LANGUAGES

- Once the design of a database is completed and a DBMS is chosen to implement the database, the first step is to specify conceptual and internal schemas for the database.
- The DBMS Languages are:
  - Data Definition Language (DDL)
  - Data Manipulation Language(DML)
  - Transaction Control Language (TCL)
  - Data Control Language(DCL)

DBMS LANGUAGES

Data Definition Language (DDL)

- Data Definition Language is used by the DBA and by database designers to define schemas.
- The DDL commands are:
  - **CREATE** - To create a new table or database
  - **ALTER** - To alter the structure of the table
  - **TRUNCATE** - To delete data from the table
  - **DROP** - To drop/delete a table
  - **RENAME** - To rename a table
- DDL commands are auto-committed

DBMS LANGUAGES

Data Manipulation Language (DML)

- Data Manipulation Language (DML) statements are used for managing data in database.
- DML provides the ability to insert tuples into, delete tuples from, and modify tuples in the database.
- The DML Commands are:
  - **INSERT** - Insert a new record into the table
  - **UPDATE** - Update the value in the existing column of the table
  - **DELETE** - Delete a record from the existing table
- DML commands are not auto-committed.

DBMS LANGUAGES

Transaction Control Language (TCL)

- Transaction Control Language (TCL) commands are used to manage transactions in the database.
- These are used to manage the changes made to the data in a table by DML statements.
- It also allows statements to be grouped together into logical transactions.
- The TCL Commands are:
  - **COMMIT** - permanently save the work done
  - **SAVEPOINT** - identify a point in a transaction to which you can later roll back
  - **ROLLBACK** - restore database to original since the last COMMIT

DBMS LANGUAGES

Data Control Language (DCL)

- Data Control Language (DCL) is used to control privileges in Database.
- To perform any operation in the database, a user needs privileges.
- The DCL commands are:
  - **GRANT** - gives user's access privileges to database
  - **REVOKE** - withdraw access privileges given with the GRANT command

DATA MODELS

- **Definition:** "*A Data model is a collection of concepts that is used to describe the structure of the database* **-** provides the necessary means to achieve this abstraction".

CATEGORIES OF DATA MODELS

- Low level or Physical model
- High level or Conceptual data model
  - ER model
- Representational or Implementation model
  - Relational Model
  - Hierarchical Model
  - Network Model
  - Object Data Model
- Self Describing Data Model
  - Semi Structured Data model

DATA MODELS

- **High-level or Conceptual data models** provide concepts that are close to the way many users perceive data.
- **Low-level or Physical data models** provide concepts that describe the details of how data is stored in the computer.
- **Representational or Implementation data models**, which provide concepts that, may be understood by end users.

DATA MODELS

1. ENTITY RELATIONSHIP MODEL (ER Model)

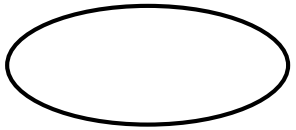
- ER Model is a Conceptual data model that use concepts such as entities, attributes, and relationships.

1. **Entity:**

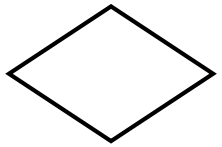
- An entity represents a real-world object or concept. Eg: a person, a bank account etc.,
- An entity is represented as 

DATA MODELS

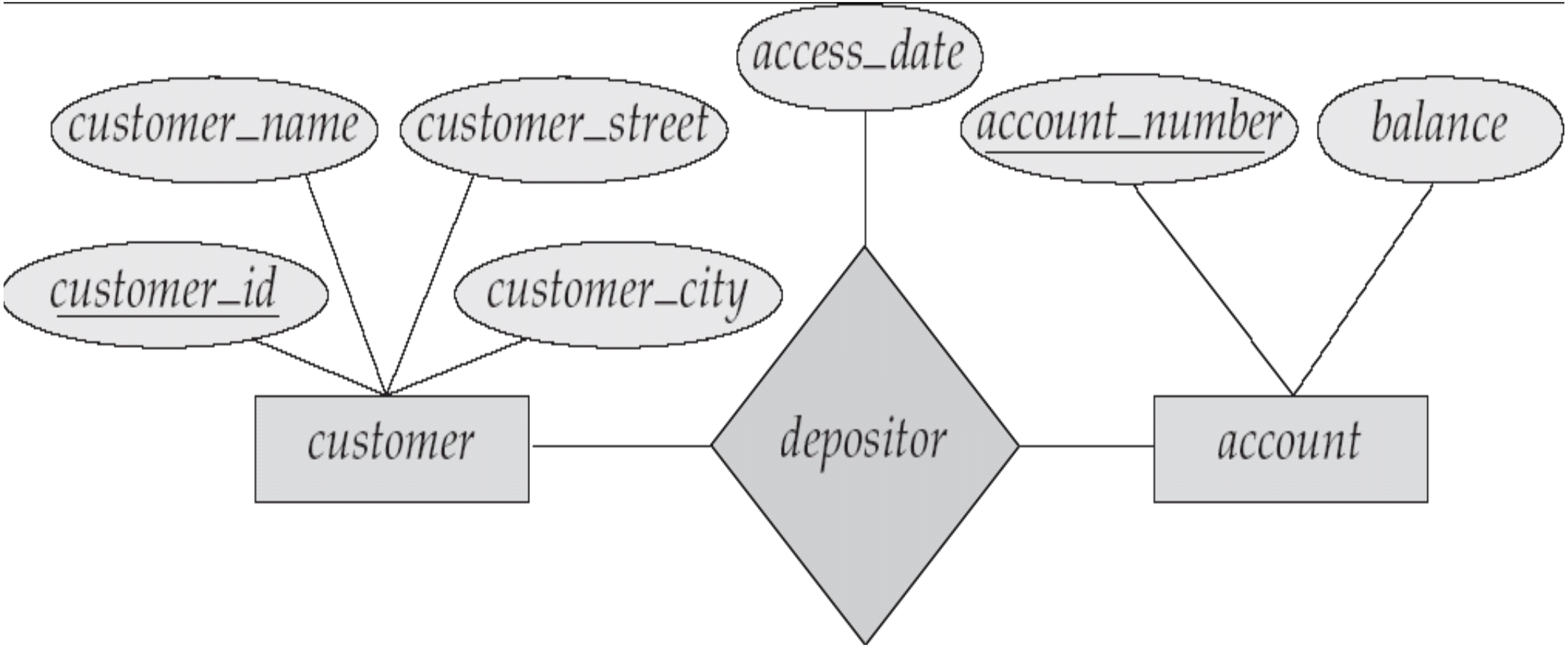
2. **Attribute:**

- An Entity can be described by a set of attributes.
- Eg: an account entity can be described by attributes acc_no, balance etc.,
- An attribute is represented as 

3. **Relationship**:

- It is an association among two or more entities.
- Eg: depositor relationship associates a customer with each account.
- A relationship set is represented as 

DATA MODELS



ER Model Example

DATA MODELS

2) RELATIONAL MODEL

- A Relational model represents data and relationship among data by a collection of tables, each of which has a number of columns with unique names.
- **Example:** Based on the above ER model, the relational model is constructed as follows:

Name	Street	City	Accountnum
John	North	Chennai	1001
Smith	South	Chennai	1002
Sri	West	Chennai	1003

***Customer table**

Accountnum	Balance
1001	80000
1002	15000
1003	24000

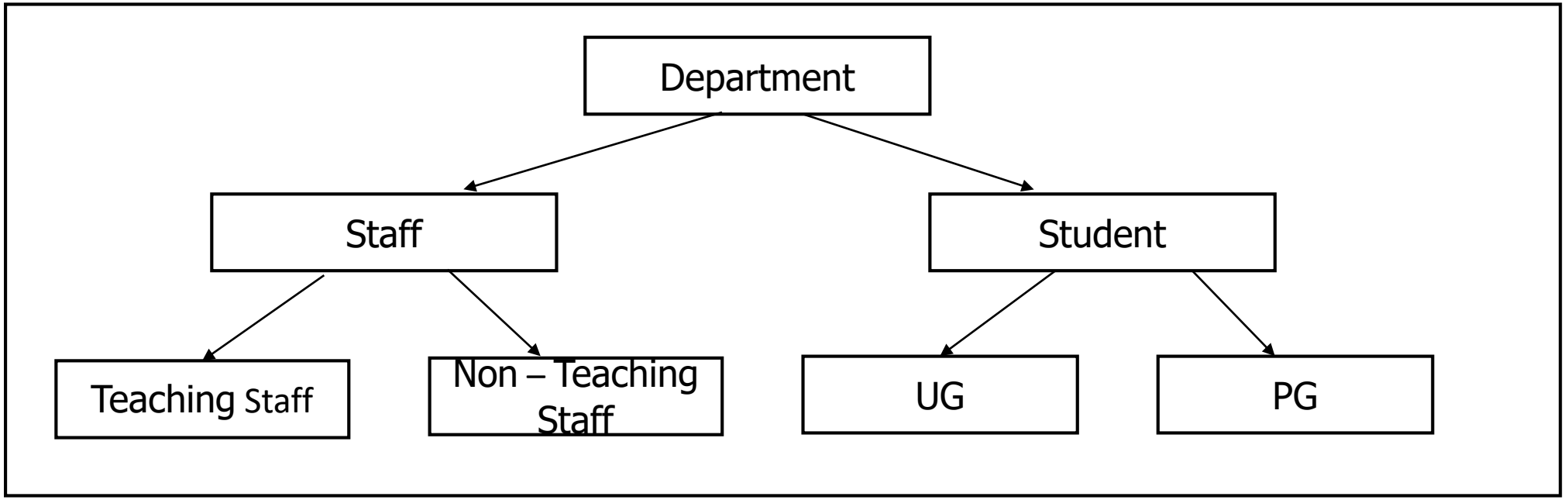
***Account table**

DATA MODELS

3) HIERARCHICAL MODEL

- A hierarchical database links records together in a tree data structure such that each record type has only one owner.
- Hierarchical data model is a way of organizing a database with multiple one to many relationships.
- The hierarchical structures were widely used in the first DBMS.

Example:

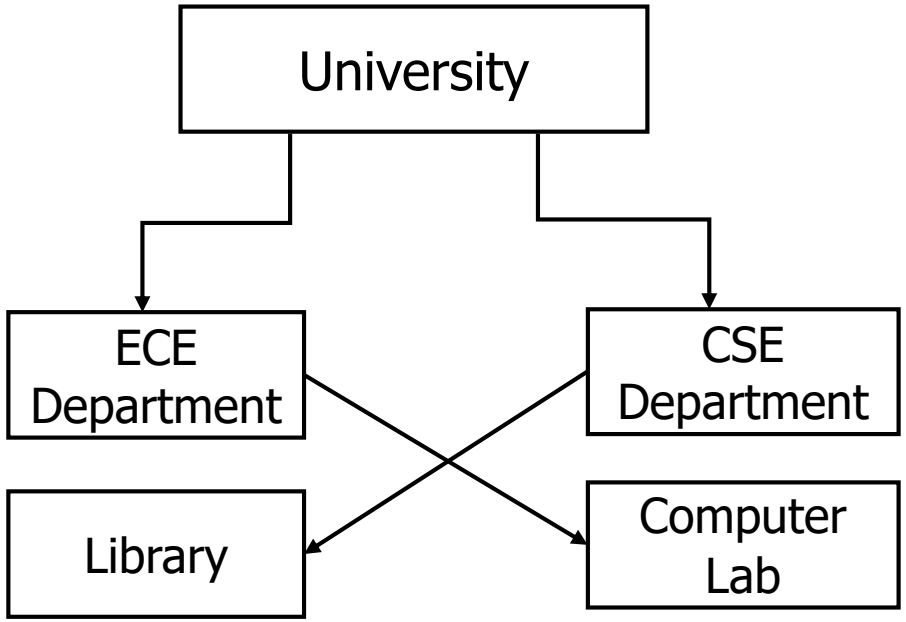


DATA MODELS

4) NETWORK MODEL

- The network model provides a flexible way of representing data objects and their relationships
- Ability to handle n:n relationships

Example:



DATA MODELS

5) OBJECT ORIENTED MODEL

- Object based data models are categorized into
  - Object Oriented Data Model
  - Object Relational Data Model
- **Object-oriented data model** can be seen as extending the ER model with notions of encapsulation, methods and object identity.
- **Object-relational data model** extends the relational data model by including object orientation and constructs to deal with added data types

DATA MODELS

6) SEMI-STRUCTURED DATA MODEL:

- The Extensible Markup Language is widely used to represent semi structured data.
- It has the ability to specify new tags, created nested tag structures, which made XML a great way to exchange data.
- A wide variety of tools is available for parsing, browsing and querying XML documents.

ENTITY RELATIONSHIP MODEL (ER MODEL)

E-R Model:

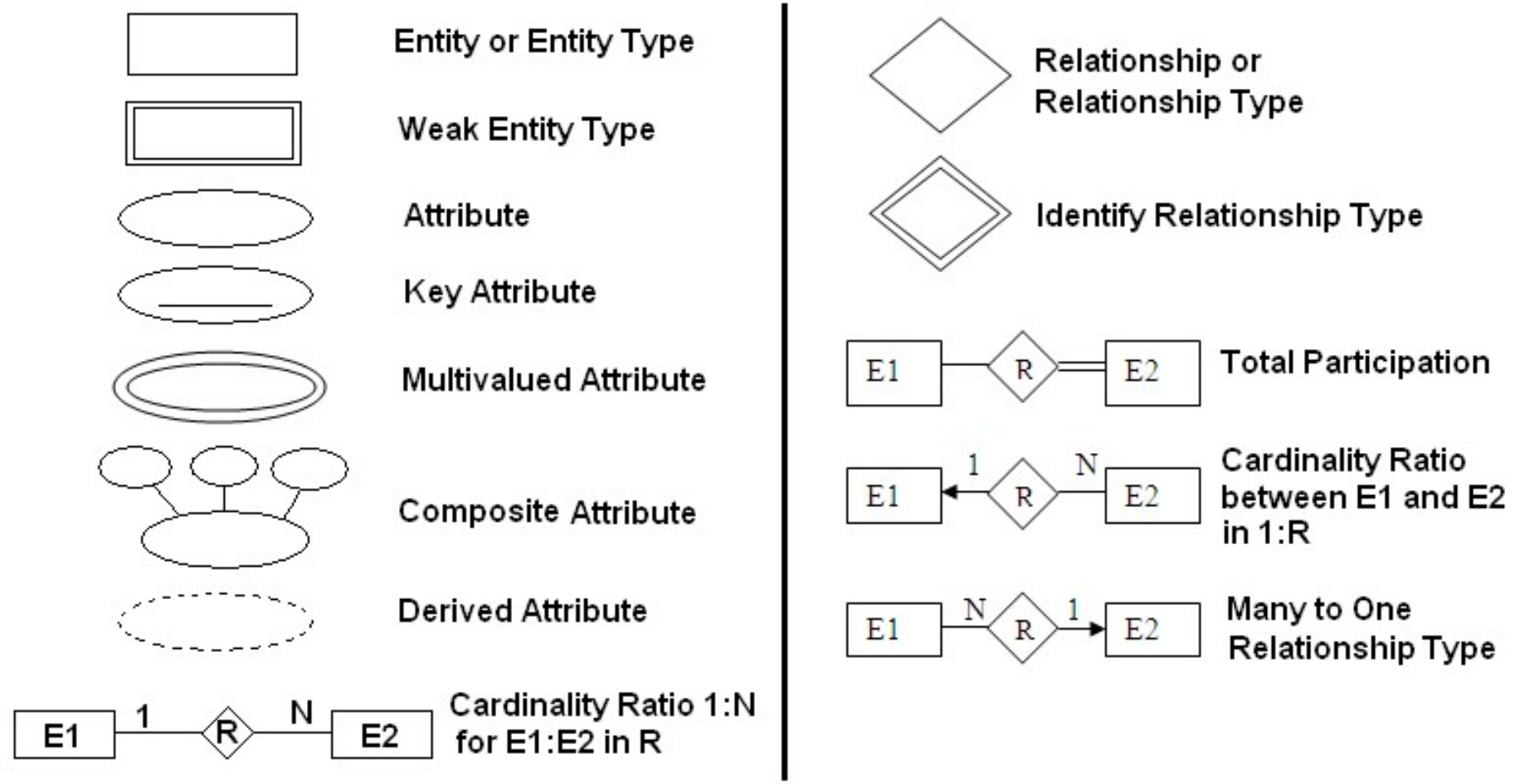
- The Entity-Relationship(E-R) data model was developed to facilitate database design.
- The E-R model is very useful in mapping the meanings and interactions of real-world enterprises onto a conceptual schema.

Basic concepts employed in E-R Data model:

1. Entity sets
2. Relationship sets
3. Attributes
4. Degree of a Relationship set
5. Weak Entity Sets
6. Constraints
    - Mapping cardinalities
    - Participation constraints
    - Keys

ENTITY RELATIONSHIP MODEL (ER MODEL)

Notations used in E-R Modeling



ENTITY RELATIONSHIP MODEL (ER MODEL)

1. Entity Sets

- **An entity** is a 'thing' or 'object' in the real world that is distinguishable from all other objects. An entity has a set of properties.
- **Example**: Each person in a university is an entity.
- **An entity set** is a set of entities of the same type that share the same properties.
- **Example:** The set of all students in the university is an entity set.

2. Relationships sets

- A relationship is an association among several entities.
- A relationship set is a set of relationships of the same type.
- **Example**: Consider two entity sets student and section. The relationship set advisor denotes the association between instructors and students.

ENTITY RELATIONSHIP MODEL (ER MODEL)

3. Attributes

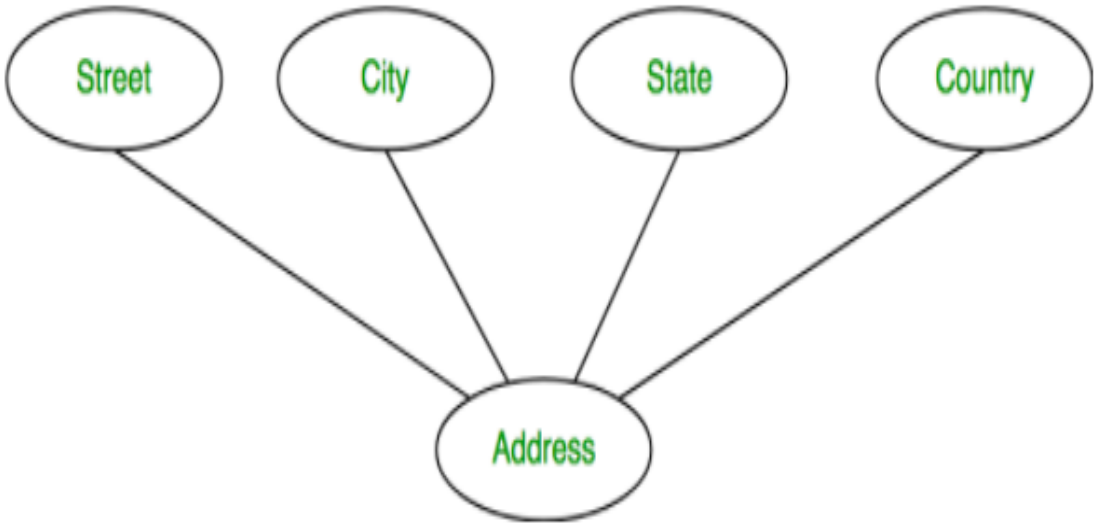
- An attribute is defined as the properties that describe an entity.
- For each attribute, there is a set of permitted values, called the ***domain*** or ***value set*** of that attribute.
- **Example:** An EMPLOYEE entity may be described by the employee's name, age, address, salary, and job.

Types of Attributes:

- **Simple Attribute:** An Attribute that is not divisible.
  - Example: Register number, Aadhar number
  - In E-R Diagram, it is represented as

ENTITY RELATIONSHIP MODEL (ER MODEL)

- **Composite Attribute:**
- An Attribute that is divisible into smaller subparts
- Example: Address.
- The address attribute can be sub-divided into street, address, city, state and pincode
- In E-R Diagram, it is represented as



ENTITY RELATIONSHIP MODEL (ER MODEL)

- **Single valued Attribute:** An Attribute that has a single value for a particular entity
  - Example: Date of birth, CGPA
- **Multi valued Attribute:** An Attribute that have a set of values for the same entity
  - Example: Phone number, email address
- In E-R Diagram, it is represented as

ENTITY RELATIONSHIP MODEL (ER MODEL)

- **Stored Attribute:** An Attribute that is already stored in the Database
  - Example: Date of birth, Date of Joining
- **Derived Attribute:** An Attribute that is derived from the stored Attribute
  - Example: Age, Experience
- In E-R Diagram, it is represented as

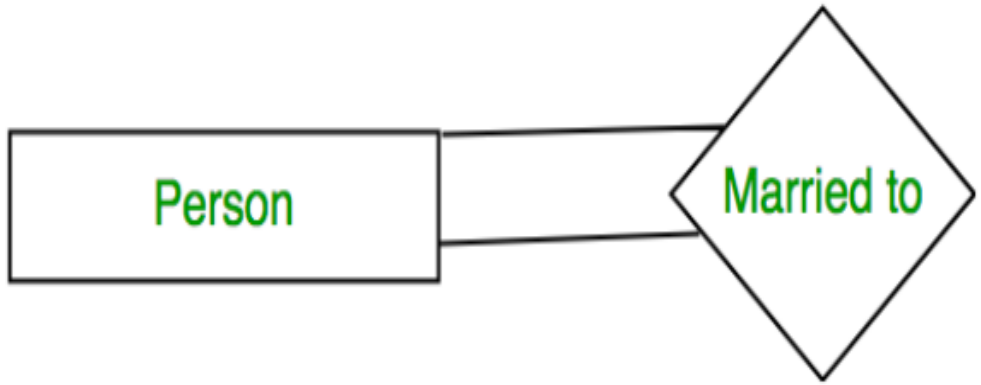
ENTITY RELATIONSHIP MODEL (ER MODEL)

4. Degree of a Relationship set

- The number of different entity sets participating in a relationship set is called as degree of a relationship set.

1. Unary Relationship

- ONE entity set participating in a relationship



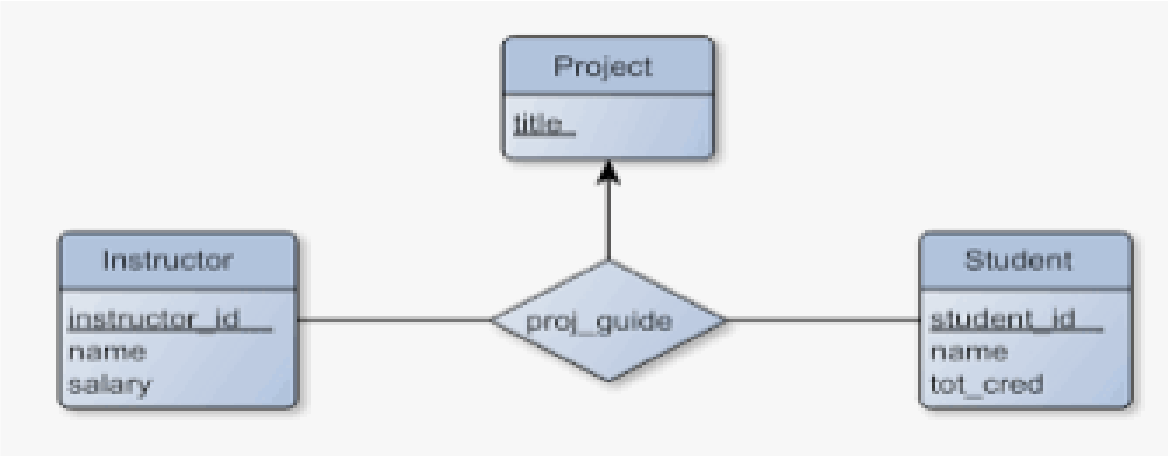
ENTITY RELATIONSHIP MODEL (ER MODEL)

2. Binary Relationship

- TWO entity set participating in a relationship

3. n-ary Relationship

- When there are n entities set participating in a relation, the relationship is called as n-ary relationship.



ENTITY RELATIONSHIP MODEL (ER MODEL)

5. Weak Entity And Strong Entity Sets

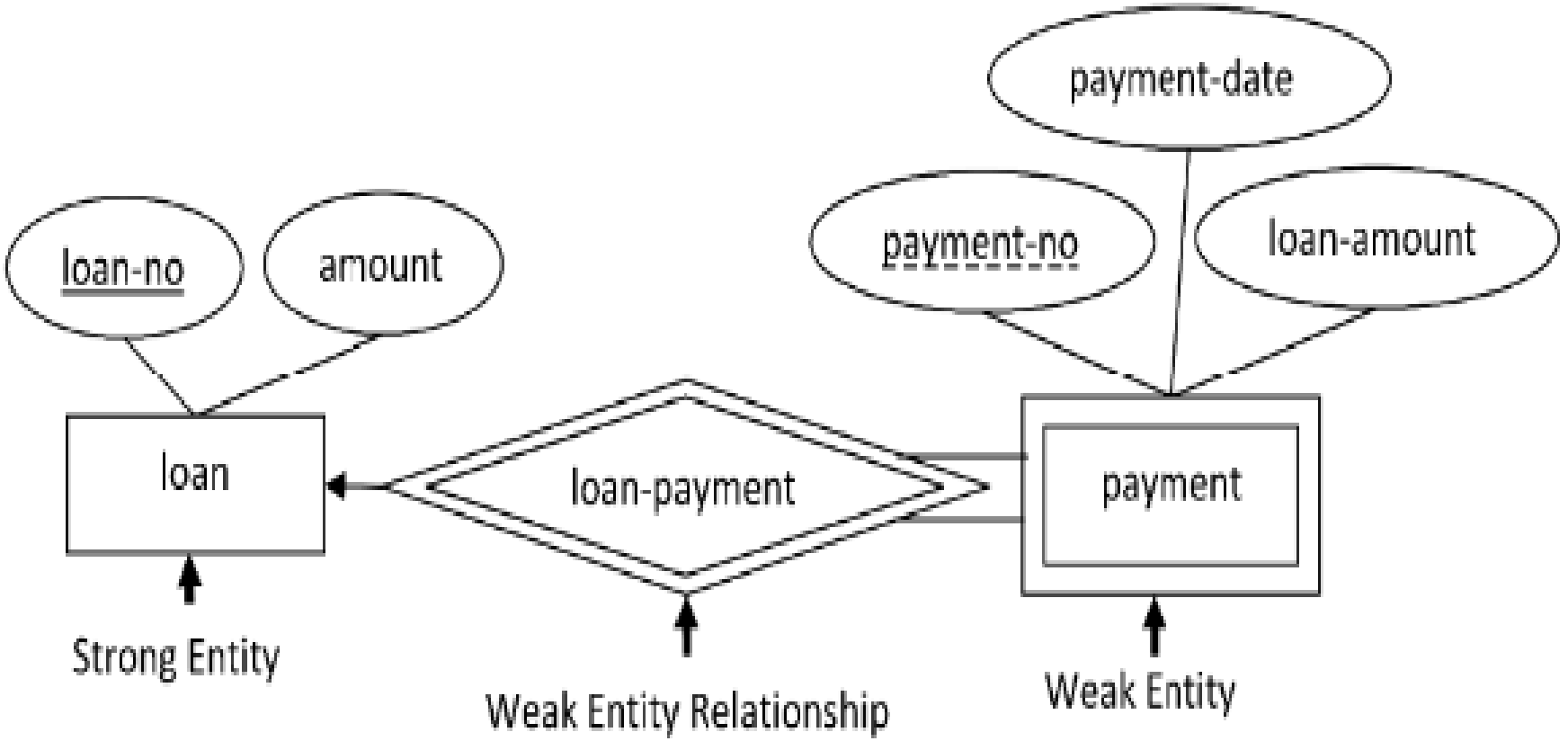
Strong Entity Set:

- An entity set that has a primary key is termed as a strong entity set.

Weak Entity Set:

- An entity set that does not have sufficient attributes to form a primary key is termed as weak entity set.
- A weak entity set must be associated with another entity set, called **identifying or owner relationship set**
- **Weak entity set must have total participation in this identifying relationship set.**
- The discriminator of the weak entity set is underlined with a dash, rather than a solid line
- The relationship set connecting the weak entity set to the identifying strong entity set is denoted by a double lined rectangle.

ENTITY RELATIONSHIP MODEL (ER MODEL)



ENTITY RELATIONSHIP MODEL (ER MODEL)

6. Constraints

- An E-R enterprise schema may define certain constraints to which the contents of the database must conform.
- The constraints in E-R model are:
  - Mapping cardinalities
  - Participation constraints
  - Keys

ENTITY RELATIONSHIP MODEL (ER MODEL)

Mapping Cardinalities:

- Mapping cardinalities/ cardinality ratios express the number of entities to which another entity can be associated via a relationship set.
- For a **binary relationship set R between entity sets A and B**, the mapping cardinality must be one of the following types:
  - **i) One to one Relationship**
  - **ii) One to many Relationship**
  - **iii) Many to one Relationship**
  - **iv) Many to many Relationship**

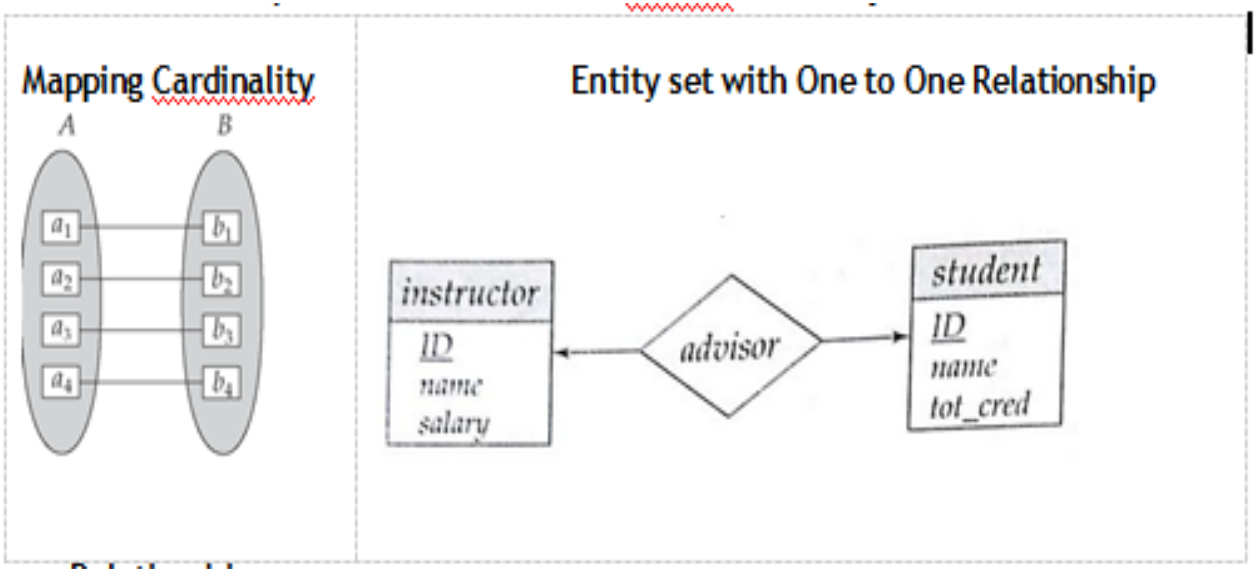
ENTITY RELATIONSHIP MODEL (ER MODEL)

i) One to one Relationship

- An entity in A is associated with **atmost** one entity in B

One-to-one relationship between an instructor and a student :

- A student is associated with at most one instructor via the relationship advisor
- A student is associated with at most one instructor



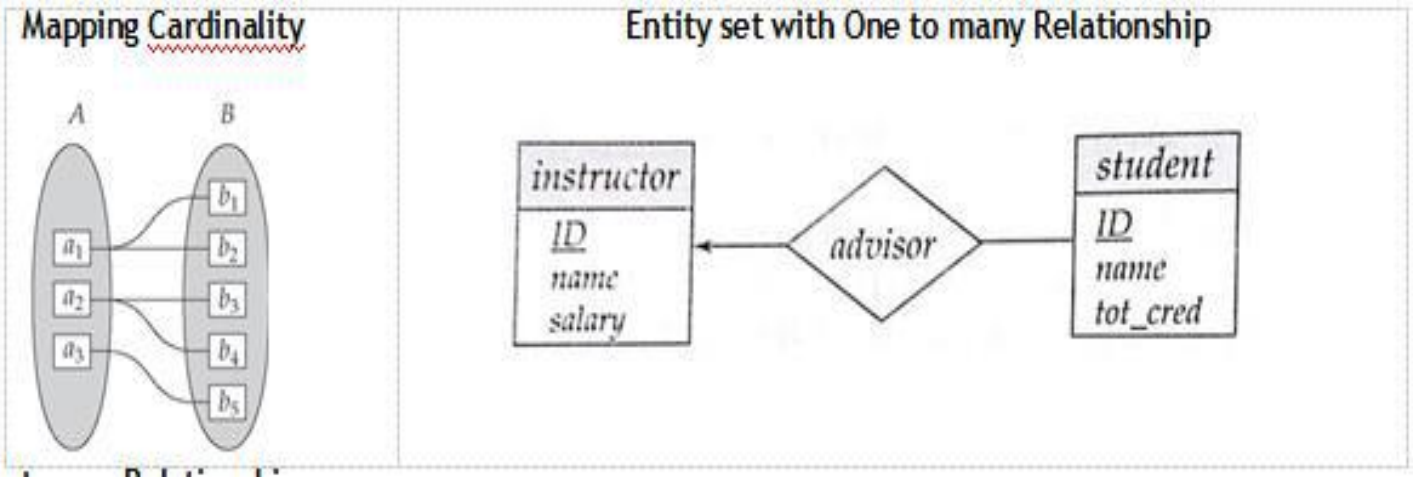
ENTITY RELATIONSHIP MODEL (ER MODEL)

ii) One to many Relationship

- An entity in A is associated with any number entities in B.
- But, an entity in B can be associated with **atmost** one entity in A.

One-to-Many relationship between an instructor and a student:

- An instructor is associated with several (including 0) students via advisor
- A student is associated with at most one instructor via advisor



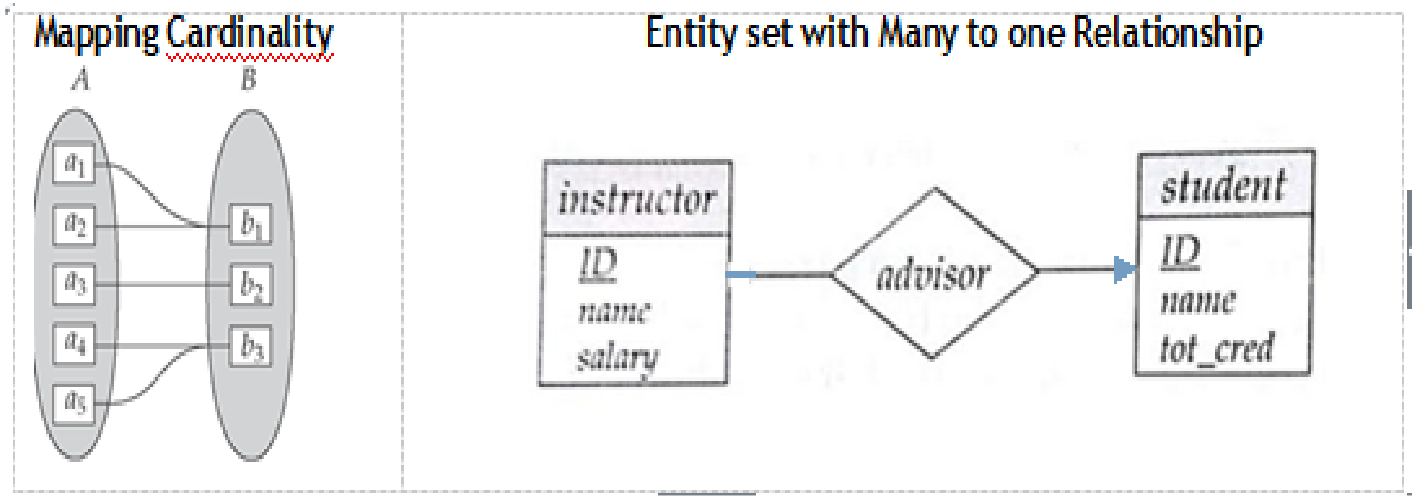
ENTITY RELATIONSHIP MODEL (ER MODEL)

iii) Many to one Relationship

- An entity in A is associated with atmost one entity in B.
- But an entity in B can be associated with any number entities in A.

In a many-to-one relationship between an instructor and a student,

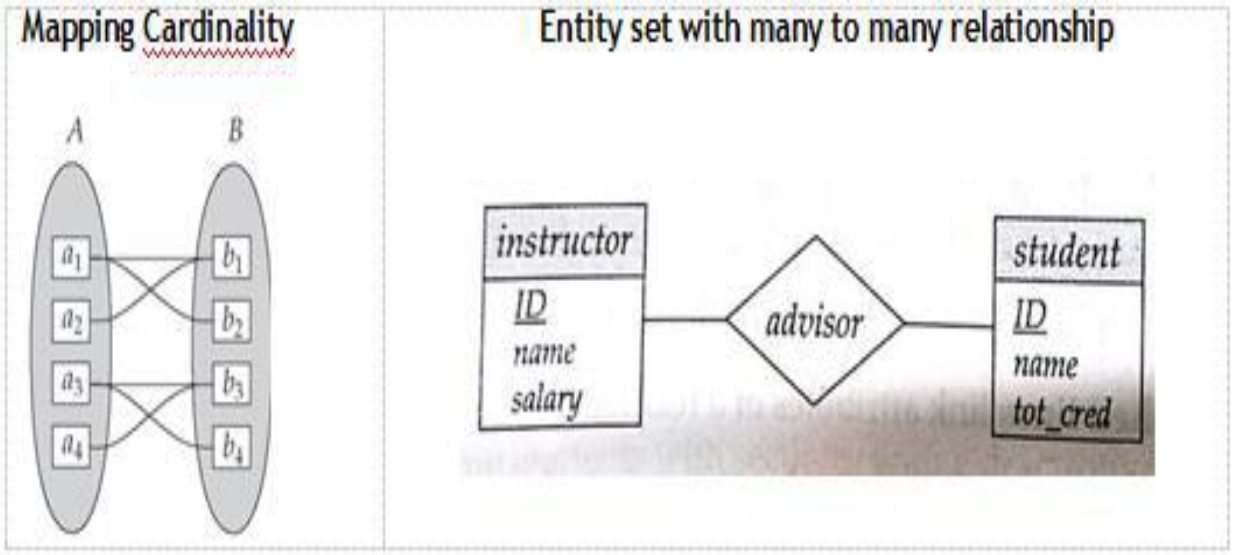
- an instructor is associated with at most one student via advisor,
- a student is associated with several (including 0) instructors via advisor.



ENTITY RELATIONSHIP MODEL (ER MODEL)

iv) Many to many Relationship

- An entity in A is associated with any number of entities in B.
- An entity in B can be associated with any number of entities in A.
- In a many-to-one relationship between an instructor and a student
- An instructor is associated with several (possibly 0) students via advisor
- A student is associated with several (possibly 0) instructors via advisor



ENTITY RELATIONSHIP MODEL (ER MODEL)

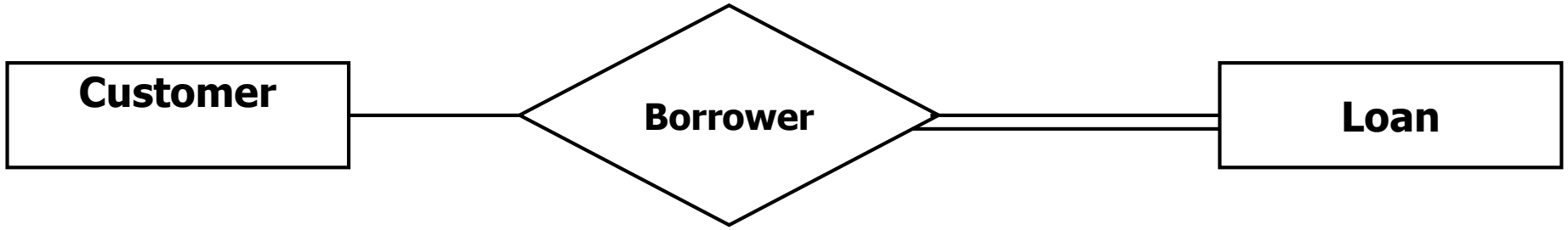
Participation Constraints:

1. Total Participation Constraint (indicated by double line)

- Every entity in the entity set participates in at least one relationship in the relationship set
- Example: participation of loan entity in borrower relationship is total.
- Every loan must have a customer associated to it via borrower.

2. Partial Participation Constraint

- Some entities may not participate in any relationship in the relationship set.
- Example: participation of customer entity in borrower relationship is partial.



ENTITY RELATIONSHIP MODEL (ER MODEL)

Keys:

- Keys specify how entities within a given entity set are distinguished. A key attribute uniquely identifies the entity.
- The concepts of super key, candidate key and primary key are applicable to entity sets just as they are applicable to relation schemas.

Types of Keys:

- Super Key
- Candidate Key
- Primary Key
- Foreign Key

ENTITY RELATIONSHIP MODEL (ER MODEL)

- A **super key** of an entity set is a set of one or more attributes whose values uniquely determine each entity.
- A **candidate key** of an entity set is a minimal super key that uniquely identifies each occurrence of an entity type. Candidate key can never be NULL.
- A **Primary Key** is the candidate key that is selected to uniquely identify an entity type. Although several candidate keys may exist, one of the candidate keys is selected to be the primary key.
- A **foreign key** is an attribute of an entity set that refers to the primary key attribute of another entity set. It enforces referential integrity constraints.

EXTENDED - ER MODEL

The extended E-R features are:

- Specialization
- Generalization
- Attribute Inheritance
- Aggregation

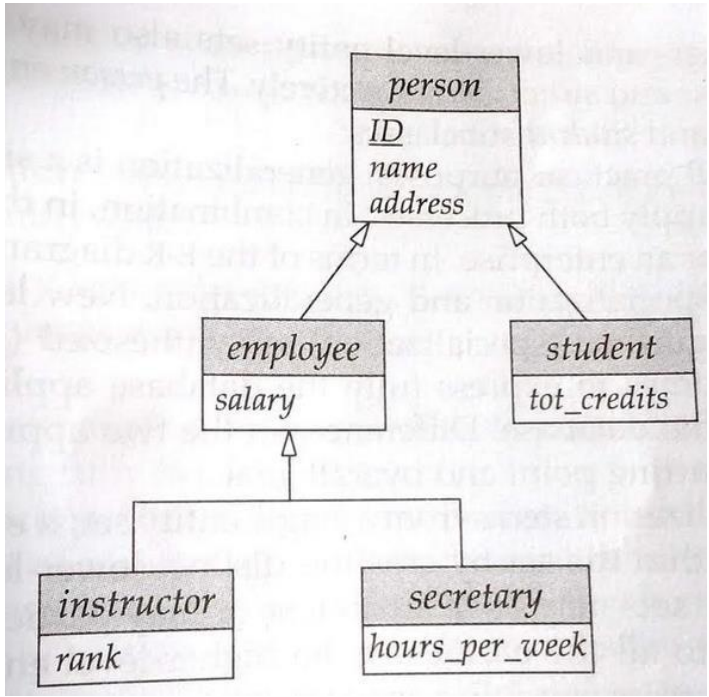
Specialization

- "Specialization is the process of designating sub groupings within an entity set."
- It is a top down design process.

EXTENDED - ER MODEL

Generalization

- "Generalization is the process in which multiple entity sets are synthesized into higher-level entity set on the basis of common features."
- It is a bottom – up design process.
- This approach results in the identification of a generalized super class from the original subclasses.



EXTENDED - ER MODEL

Generalization Constraints

1) "To determine which entities can be members of a given Lower-level entity set."

- **Condition Defined or predefined:**
  - In condition-defined lower-level entity sets, membership is evaluated on the basis of whether or not an entity satisfies an explicit condition or predicate.
- **User Defined:**
  - User defined lower-level entity sets are not constrained by a membership condition.
  - The database user assigns entities to a given entity set.

EXTENDED - ER MODEL

2) "To determine whether or not entities may belong to more than one lower-level entity set."

- **Disjoint:**
  - A disjoint generalization requires that an entity belong to no more than one lower –level entity set.
- **Overlapping:**
  - In Overlapping generalizations, the same entity may belong to more than one lower-level entity set.

EXTENDED - ER MODEL

3) Completeness constraint on Generalization or specialization:

"To determine whether or not an entity in the higher-level entity set must belong to atleast one of the lower-level entity set within the generalization or specialization."

- **Total Generalization:**
  - Each higher-level entity must belong to a lower-level entity set.
- **Partial Generalization:**
  - Some higher-level entities may not belong to any lower-level entity set.

EXTENDED - ER MODEL

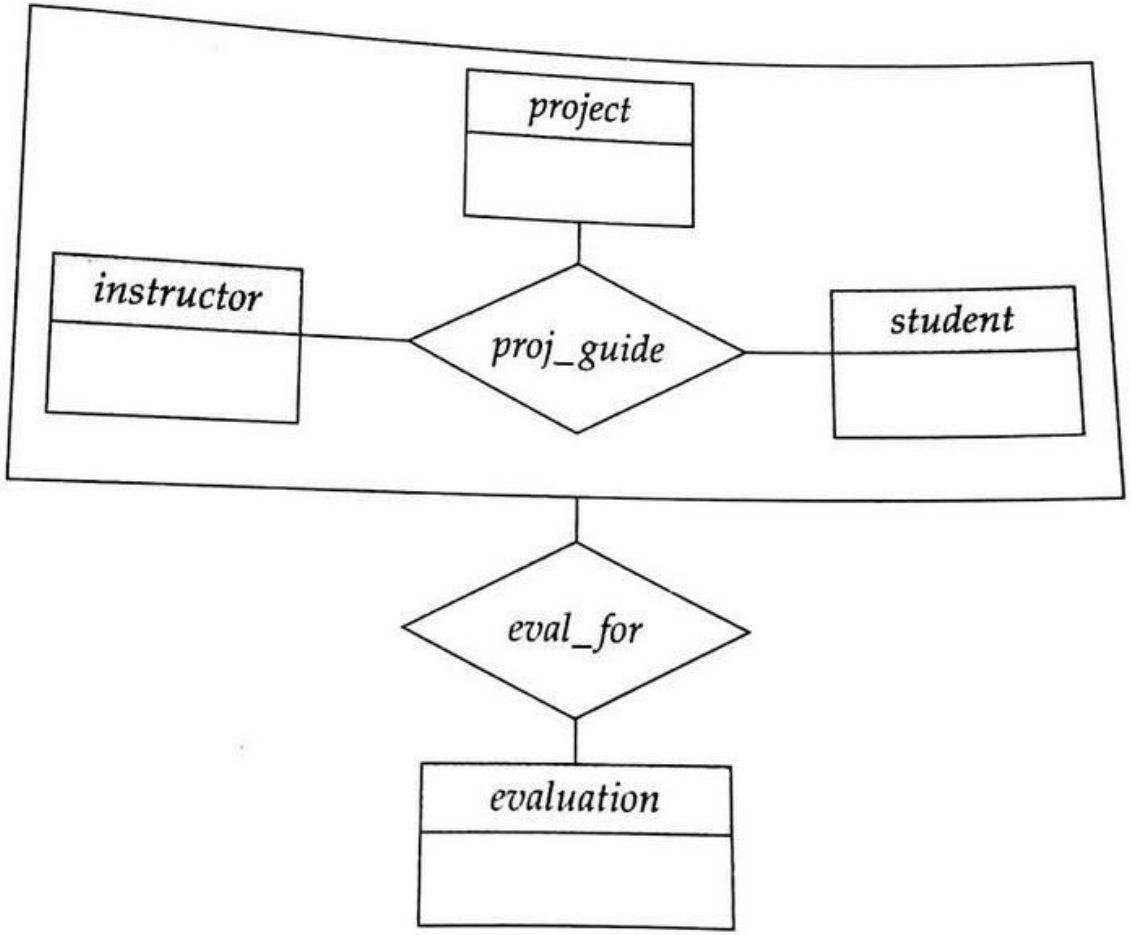
Attribute Inheritance:

- A crucial property of the higher and lower level entities created by specialization and generalization is attribute inheritance.
- The attributes of the higher-level entity sets are said to be inherited to the lower-level entity sets.

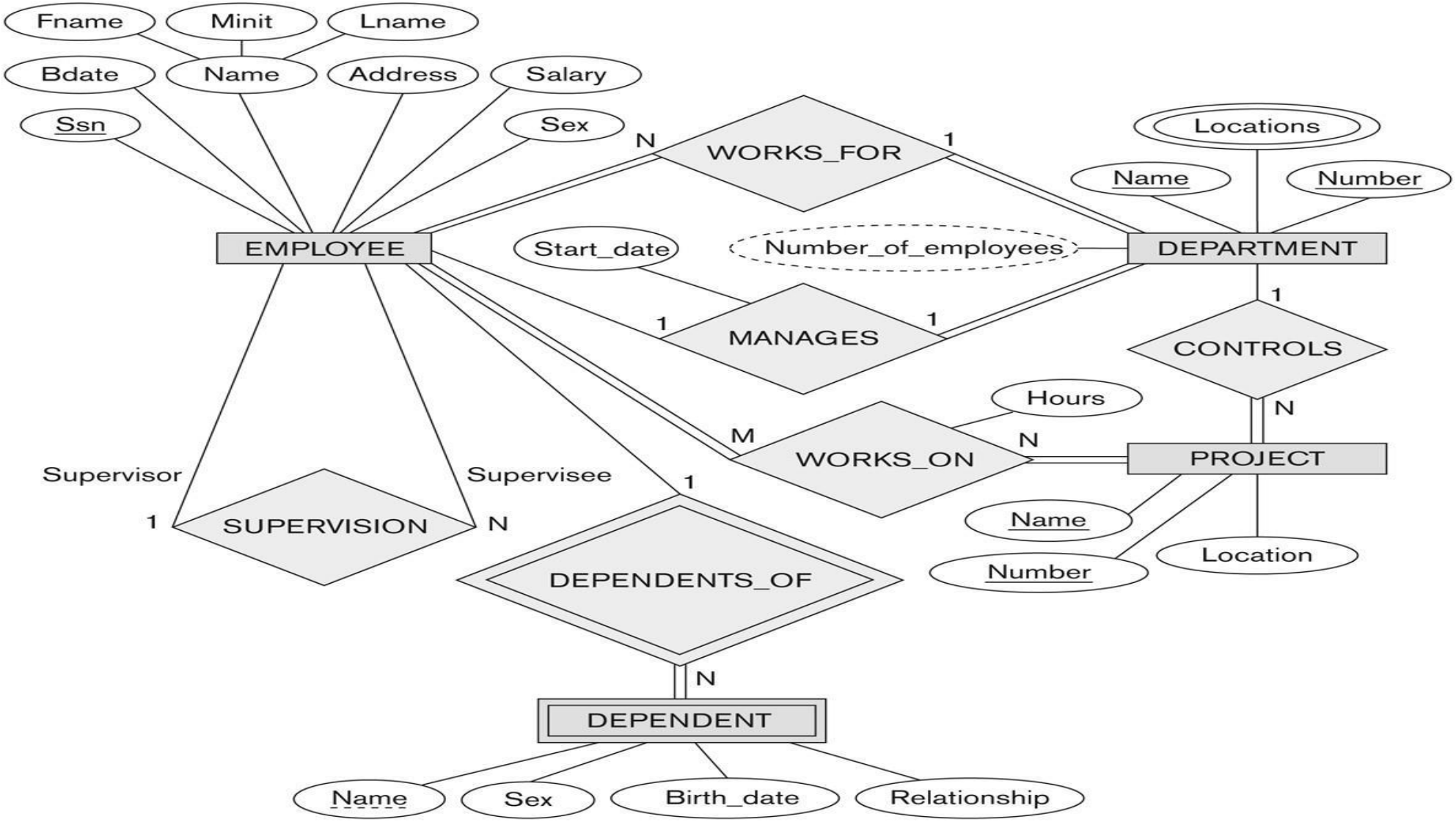
Aggregation:

- Aggregation is an abstraction through which relationships are treated as higher level entities.
- It is used when we have to model a relationship involving (entity sets and) a relationship set.
- Aggregation allows us to **treat a relationship set as an entity set** for purposes of participation in (other)relationships.

EXTENDED - ER MODEL



EXTENDED - ER MODEL



E-R Diagram for company database

REDUCING ER DIAGRAM TO RELATIONAL SCHEMAS

- Conversion of ER diagram to relational schemas is a SEVEN step process

1. Mapping of Regular entity types
2. Mapping of Weak entity types
3. Mapping of binary 1:1 relationship types
4. Mapping of binary 1:n and n:1 relationship types
5. Mapping of binary m:n relationship types
6. Mapping Multi-valued attributes
7. Mapping n-ary relationship types

REDUCING ER DIAGRAM TO RELATIONAL SCHEMAS

Step1: Mapping of Regular Entity Types

- For each strong/regular entity type, create a corresponding relation that includes all simple attributes.
- If it is a composite attribute, all simple attributes of the composite attribute are included in the relation.
- Choose a primary key attribute for the relation.

Step 2: Mapping of Weak Entity Types

- For each weak entity type, create a corresponding relation that includes all simple attributes.
- Add the primary key of owner entity type as a foreign key in the relation created for the weak entity type.
- The primary key for this corresponding relation is the combination of the primary key of the owner entity type and the partial key of the weak entity type.

REDUCING ER DIAGRAM TO RELATIONAL SCHEMAS

Step 3: Mapping binary 1:1 Relationship types

- Choose one relation (table) as S and other as T (Better if S has total participation)
- Add to S all the simple attributes of the relationship
- Add the primary key attribute of T as a foreign key in S.

Step 4: Mapping binary 1:n and n:1 Relationship types

- Choose the relation (table) on the n-side of the relationship as S, Other as T.
- Add to S all the simple attributes of the relationship
- Add the primary key attribute of T as a foreign key in S.

REDUCING ER DIAGRAM TO RELATIONAL SCHEMAS

Step 5: Mapping binary m:n Relationship types

- Create a new relation (table) S. Table S is termed as relationship relation.
- Add to S all simple attributes of the relationship.
- Add the primary keys of both the relations as the foreign keys in S. Their combination forms the primary key of S.

Step 6: Mapping multivalued attributes

- Create a new relation S.
- The primary key of the corresponding relation is added as a foreign key.
- Add the multivalued attribute to S.
- The combination of all attributes in S forms the primary key.

REDUCING ER DIAGRAM TO RELATIONAL SCHEMAS

Step 7: Mapping of n-ary relationship types

- Create a new relation (table) S to represent the relationship.
- Add to S all simple attributes of the relationship.
- Add the primary keys of participating relations as the foreign keys in S.

ER model of University Database Application

Entity Types for University Database applications may be

- Instructor
- Department
- Student
- Course
- Dependent (Weak entity set)
- Entity Type: Instructor
  - **Attributes**: id,name, dept, salary
  - **Primary key**: id
- Entity Type: Department
  - **Attributes**: deptname, building, budget
  - **Primary key** : deptname

ER model of University Database Application

- Entity Type: Student
  - **Attributes**: regno, phone, name, totalcredit, cgpa
  - **Primary key**: regno
- Entity Type: course
  - **Attributes**: courseid, title, credits
  - **Primary key**: courseid
- Weak Entity Type: Dependent
  - **Attributes**: name, relationship, location
  - **Partial key**: name

ER model of University Database Application

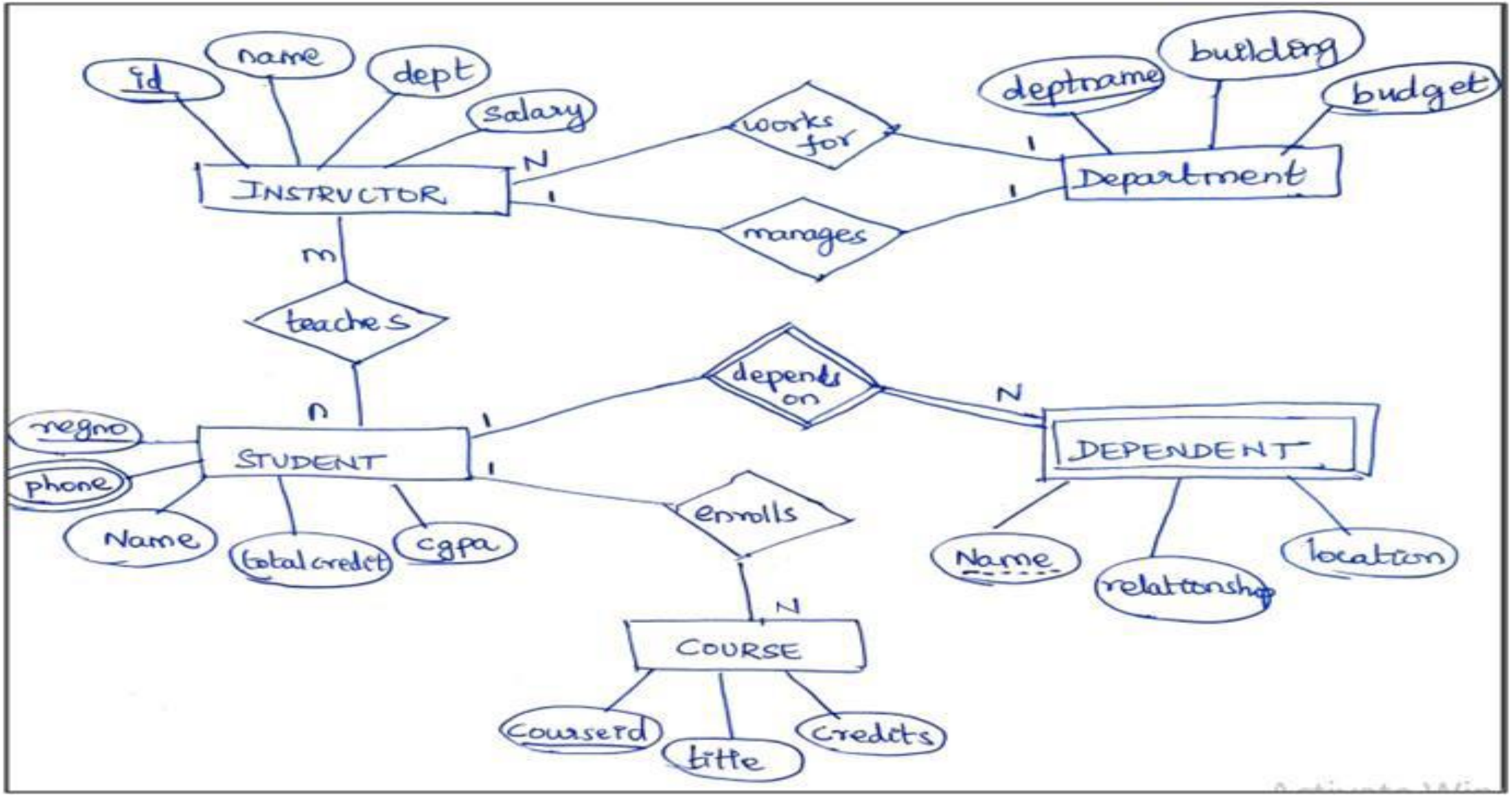
Relationship sets for University Database applications may be

- Relationship set: worksfor
  - **Participating entity types**: instructor and department
  - **Mapping cardinality**: many to one
- Relationship set: manages
  - **Participating entity types**: instructor and department
  - **Mapping cardinality**: one to one
- Relationship set: teaches
  - **Participating entity types**: instructor and student
  - **Mapping cardinality**: many to many

ER model of University Database Application

- Relationship set: enrolls
  - **Participating entity types**: student and course
  - **Mapping cardinality**: one to many
- Relationship set: depends on
  - **Participating entity types:** student and dependent
  - **Mapping cardinality**: one to many

ER model of University Database Application



ER diagram for University Database Application

Relational Database Design by ER and EER-to-Relational Mapping

Mapping of Regular Entity Types

- For each strong/regular entity type, create a corresponding relation that includes all simple attributes.
- If it is a composite attribute, all simple attributes of the composite attribute are included in the relation.
- Choose a primary key attribute for the relation.

INSTRUCTOR

id	Name	Dept	salary

DEPARTMENT

Deptname	Building	budget

STUDENT

Regno	Name	totalcredit	cgpa

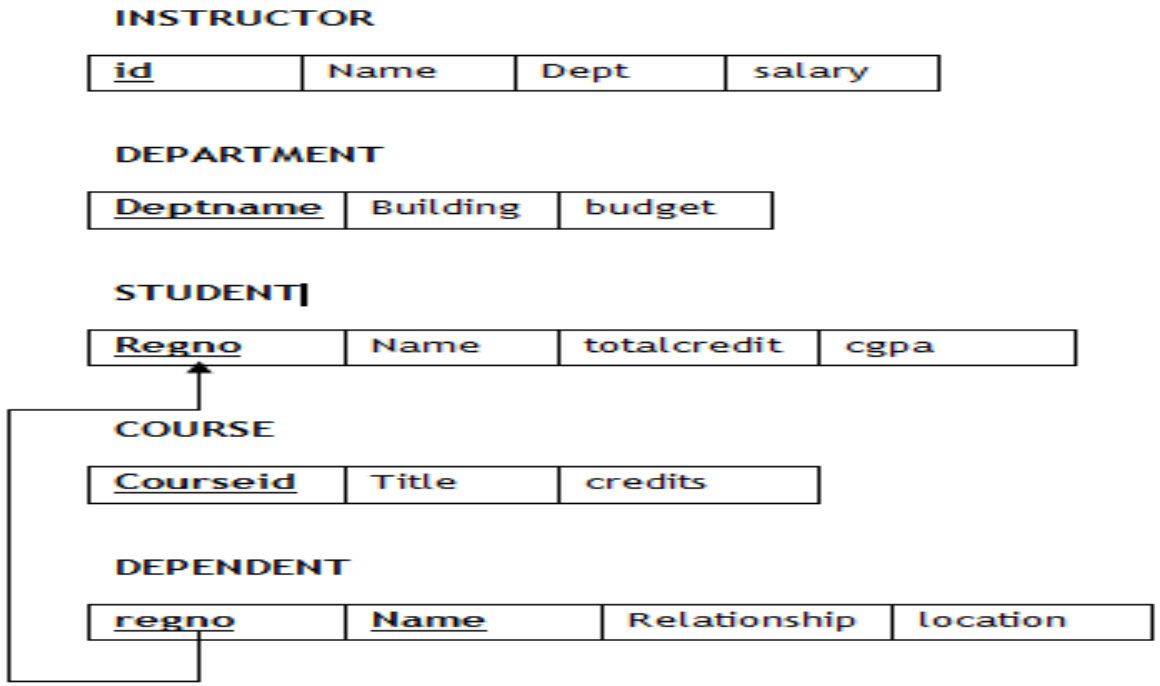
COURSE

Courseid	Title	credits

Relational Database Design by ER and EER-to-Relational Mapping

Mapping of Weak Entity Types

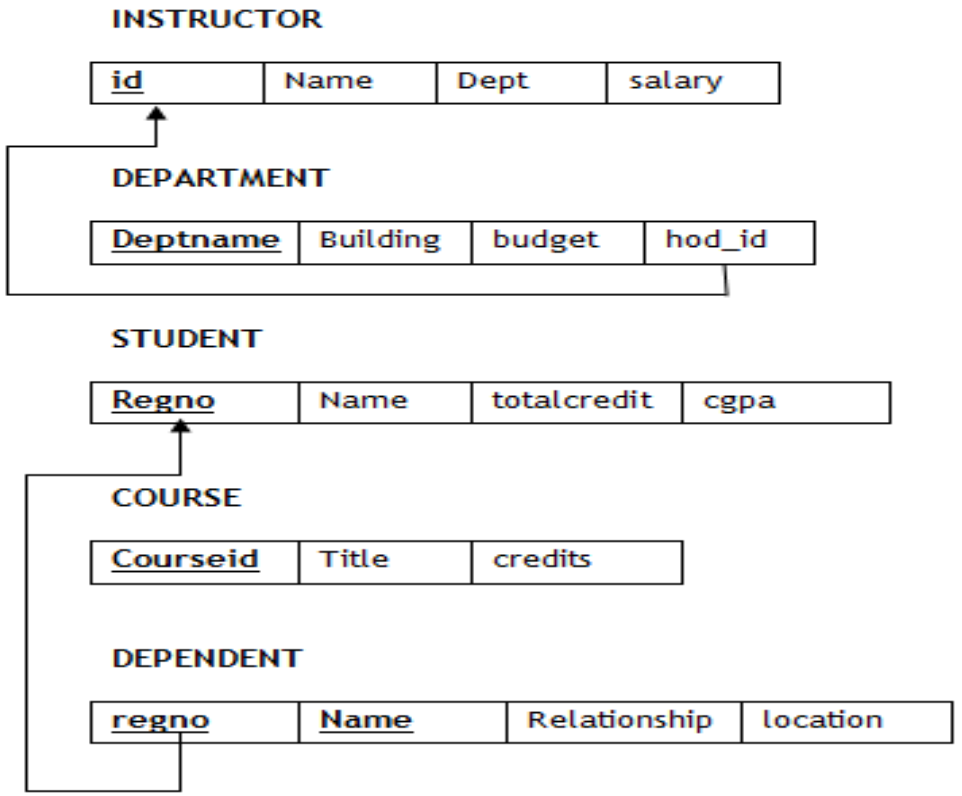
- For each weak entity type, create a corresponding relation that includes all simple attributes.
- Add the primary key of owner entity type as a foreign key in the relation created for the weak entity type.
- The primary key for this corresponding relation is the combination of the primary key of the owner entity type and the partial key of the weak entity type.



Relational Database Design by ER and EER-to-Relational Mapping

Mapping binary 1:1 Relationship types

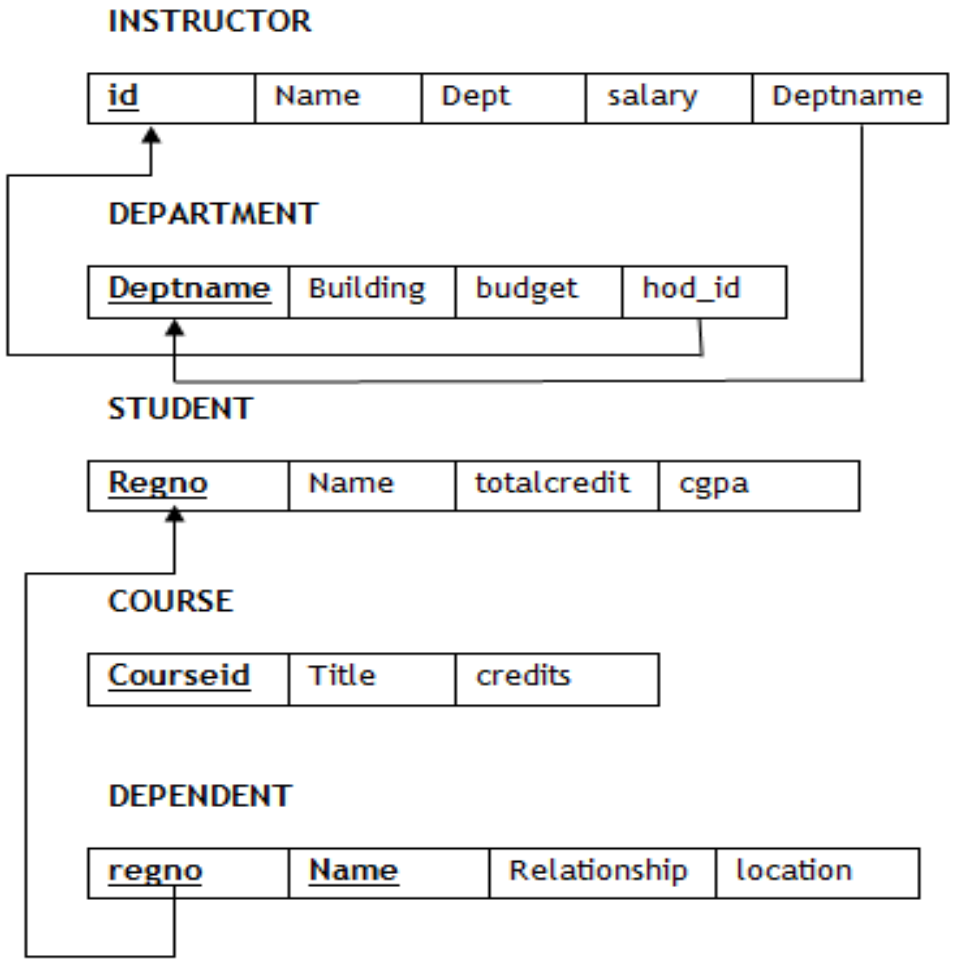
- Choose one relation (table) as S and other as T (Better if S has total participation)
- Add to S all the simple attributes of the relationship
- Add the primary key attribute of T as a foreign key in S.



Relational Database Design by ER and EER-to-Relational Mapping

Mapping binary 1:n and n:1 Relationship types

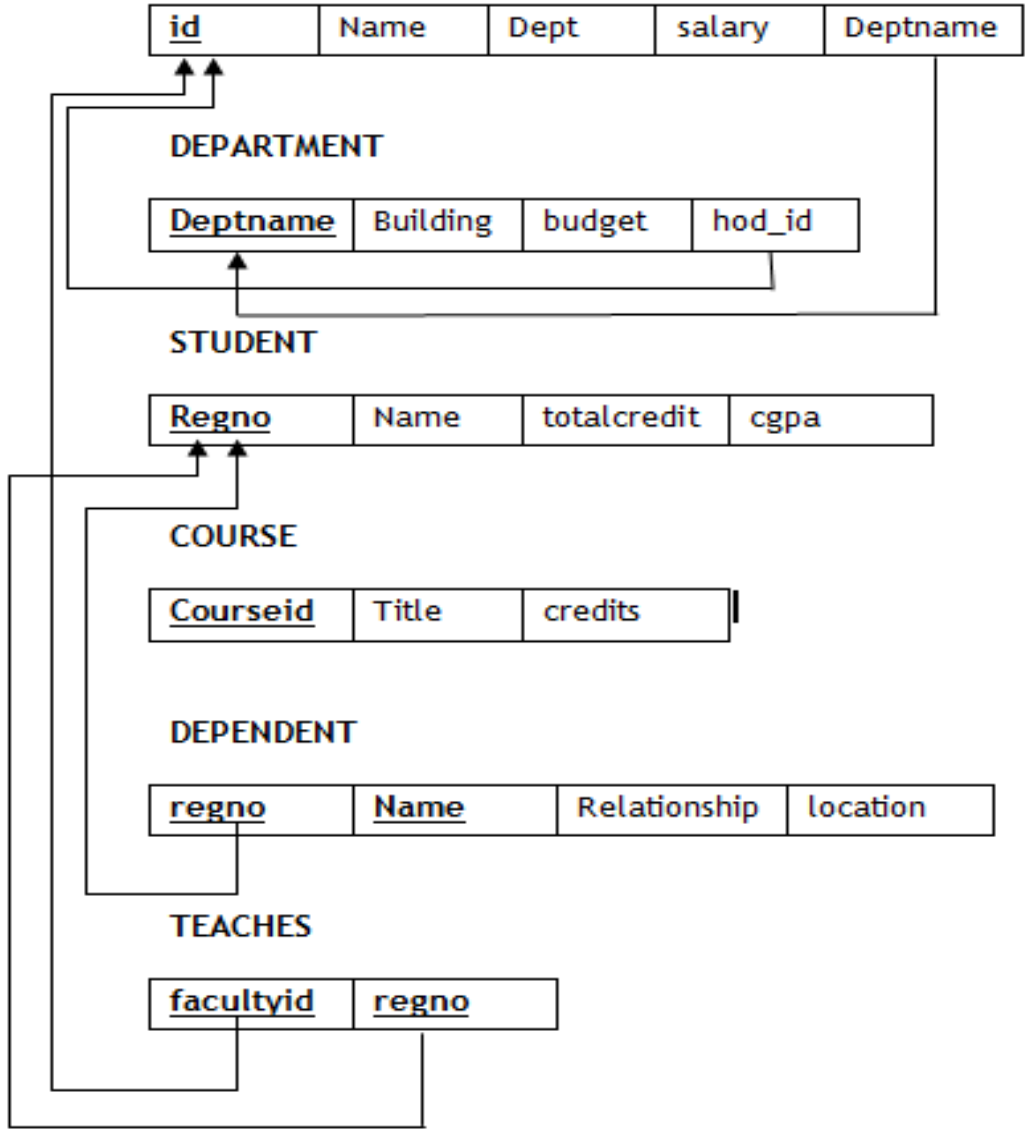
- Choose the relation (table) on the n-side of the relationship as S, Other as T.
- Add to S all the simple attributes of the relationship
- Add the primary key attribute of T as a foreign key in S.



Relational Database Design by ER and EER-to-Relational Mapping

Mapping binary m:n Relationship types

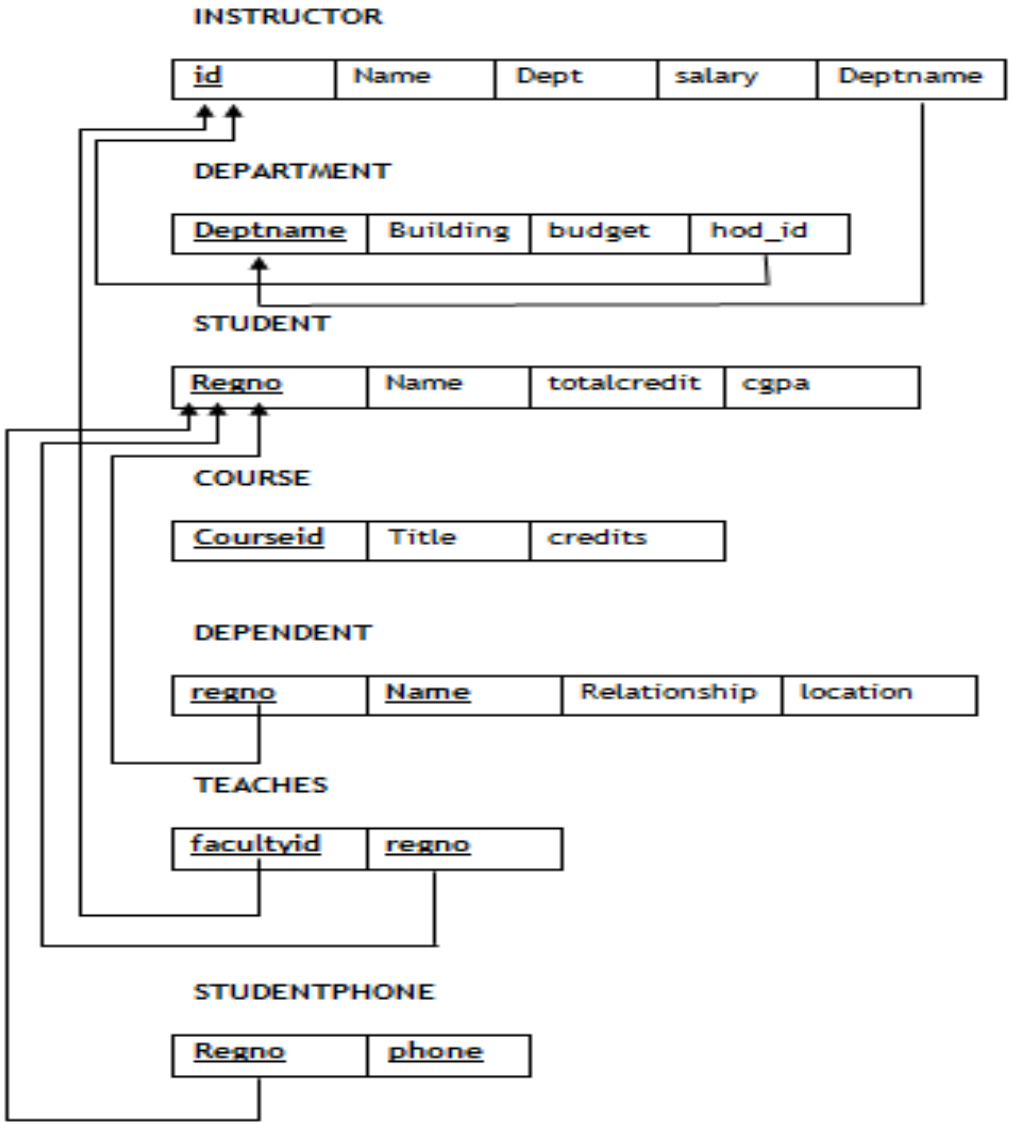
- Create a new relation (table) S. Table S is termed as relationship relation.
- Add to S all simple attributes of the relationship.
- Add the primary keys of both the relations as the foreign keys in S. Their combination forms the primary key of S.



Relational Database Design by ER and EER-to-Relational Mapping

Mapping multivalued attributes

- Create a new relation S.
- The primary key of the corresponding relation is added as a foreign key.
- Add the multivalued attribute to S.
- The combination of all attributes in S forms the primary key.



THANK YOU